

Database Programming with JDBC

Connecting Java Applications to Data

Learning Objectives

- **What We Will Learn**

- Understand the JDBC Architecture.
- Load Drivers and Establish Connections (DriverManager).
- Execute SQL Queries (SELECT) and Updates (INSERT, UPDATE, DELETE).
- Process Results using ResultSet.
- Prevent SQL Injection using PreparedStatement.
- Manage Transactions (Commit/Rollback).
- Integrate JDBC with Swing (Threading & DAO Pattern).
- Handle SQLException properly.

What is JDBC?

- Java Database Connectivity (JDBC)
 - Definition: A Java API that defines how a Java application accesses a database.
 - Architecture:
 1. Java Application
 2. JDBC API (java.sql package)
 3. JDBC Driver (Vendor-specific, e.g., MySQL, PostgreSQL)
 4. Database
 - Role: Acts as a bridge/translator between Java code and SQL database.

Setup & Drivers

- **Title:** Setting Up JDBC
 - **JDBC Driver:** Not included in standard JDK. Must be added separately.
 - **MySQL:** mysql-connector-j
 - **PostgreSQL:** postgresql
 - **SQLite:** sqlite-jdbc
 - **Loading Drivers:**
 - **JDBC 4+ (Modern):** Auto-loaded via classpath (no code needed).
 - **Legacy:** `Class.forName("com.mysql.cj.jdbc.Driver")`.

```
String url = "jdbc:mysql://localhost:3306/mydatabase";  
String user = "root";  
String password = "password";
```

Establishing a Connection

- **The Connection Object**
 - **Entry Point:** DriverManager.getConnection().
 - **Resource Management:** Always close connections to avoid leaks.
 - **Best Practice:** Use Try-With-Resources.

```
String url = "jdbc:mysql://localhost:3306/shop";
try (Connection conn =
DriverManager.getConnection(url, user, pass)) {
    // Use connection
} catch (SQLException e) {
    e.printStackTrace();
}
// Connection automatically closed here
```

Statement vs. PreparedStatement

- Executing SQL
 - Statement:
 - Used for static SQL strings.
 - **Vulnerable to SQL Injection.**

```
Statement stmt = conn.createStatement();  
stmt.execute("SELECT * FROM users WHERE id = 1");
```

- PreparedStatement:
 - Used for dynamic SQL (parameters).
 - **Prevents SQL Injection.**
 - **Better Performance** (pre-compiled)

```
PreparedStatement pstmt = conn.prepareStatement("SELECT  
* FROM users WHERE id = ?");  
pstmt.setInt(1, 1);
```

Querying Data (SELECT)

- Retrieving Data with ResultSet
- Method: executeQuery() returns a ResultSet.
- Cursor: Initially positioned before the first row.
- Navigation: Use next() to move cursor.
- Extraction: getString(), getInt(), etc.

```
String sql = "SELECT id, name FROM products";
try (PreparedStatement pstmt =
    conn.prepareStatement(sql);
    ResultSet rs = pstmt.executeQuery()) {
    while (rs.next()) {
        int id = rs.getInt("id");
        String name = rs.getString("name");
        System.out.println(id + ": " + name);
    }
}
```

Updating Data (INSERT/UPDATE/DELETE)

- **Title:** Modifying Data
- **Method:** executeUpdate() returns an int (rows affected).
- **Operations:** INSERT, UPDATE, DELETE, CREATE.

```
String sql = "INSERT INTO users (name, email) VALUES (?, ?)";
try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
    pstmt.setString(1, "Alice");
    pstmt.setString(2, "alice@example.com");
    int rows = pstmt.executeUpdate();
    System.out.println(rows + " row(s) inserted");
}
```


SQL Injection Attack

- Security: SQL Injection
- Vulnerability: Occurs when user input is concatenated into SQL.
- Attack Example:
 - Input: ' OR '1'='1
 - SQL: SELECT * FROM users WHERE name = " OR '1'='1'
 - Result: Returns all users (bypasses login).
- Prevention: Use PreparedStatement.
 - Treats input as data, not code.
 - Escapes special characters automatically.

Transactions

- Managing Transactions
 - **Definition:** A group of SQL operations treated as a single unit.
 - **ACID Properties:** Atomicity, Consistency, Isolation, Durability.
 - **Control:**
 - `conn.setAutoCommit(false)`
 - `conn.commit()` (Save changes)
 - `conn.rollback()` (Undo changes on error)
 - **Use Case:** Transferring money (Debit A + Credit B must both succeed).

```
conn.setAutoCommit(false);  
try {  
    // execute updates  
    conn.commit();  
} catch (SQLException e) {  
    conn.rollback();  
}
```

JDBC in Swing Applications

- Integrating Database with GUI
 - Problem: Database operations are blocking I/O.
 - Rule: NEVER run JDBC code on the Event Dispatch Thread (EDT).
 - Result: UI freezes while waiting for DB response.
 - Solution: Use `SwingWorker`.
 - `doInBackground()`: Run JDBC query.
 - `done()`: Update `JTable` or `JLabel`.

The DAO Pattern

- Data Access Object (DAO) Pattern
 - **Problem:** Mixing SQL code with Swing UI code creates "Spaghetti Code".
 - **Solution:** Separate concerns.
 - **View:** Swing Components (JFrame).
 - **Model:** Data Objects (User class).
 - **DAO:** Database Logic (UserDAO class).
 - **Benefit:** Easier to test, maintain, and swap databases.

```
// UserDAO.java
public class UserDAO {
    public void save(User u) { ... } // JDBC code here
    public List<User> getAll() { ... }
}
```

Connection Pooling

- **Performance: Connection Pooling**
- **Problem:** Creating a new Connection is slow (handshake, authentication).
- **Solution:** Reuse existing connections from a "Pool".
- **Libraries:** HikariCP, Apache DBCP.

```
// Instead of DriverManager
DataSource ds = HikariConfig.getDataSource();
try (Connection conn = ds.getConnection()) { ... }
```

- **Note:** Essential for high-traffic web apps; optional for simple desktop apps.

Common Pitfalls

- **Title:** JDBC Best Practices
 - ☒ **Do:**
 - Use PreparedStatement always.
 - Use Try-With-Resources for Connection, Statement, ResultSet.
 - Handle SQLException specifically.
 - Store passwords securely (not in code!).
 - ☒ **Don't:**
 - Hardcode DB credentials in source control.
 - Run DB queries on the EDT.
 - Keep Connections open longer than necessary.
 - Ignore warnings from the database.

Summary & Lab

- **Summary:**
 - JDBC connects Java to SQL databases.
 - Use PreparedStatement for security.
 - Manage transactions for data integrity.
 - Keep DB logic separate from UI (DAO).
 - Run DB tasks in background threads.
- **Lab Exercise:**
 - Task: Create a "Student Manager" Swing App.
 - Database: MySQL/SQLite table students (id, name, grade).
 - Features:
 - Add Student (INSERT).
 - View All Students (SELECT into JTable).
 - Delete Student (DELETE).